Segment Tree & Lazy Segment Tree

ACL Quick-Reference Cheatsheet

How to Include the AtCoder Library (ACL)

Option 1 — On AtCoder judge (nothing needed)

ACL is pre-installed on the AtCoder judge. Just include the header:

```
#include <atcoder/segtree>       // plain segment tree
#include <atcoder/lazysegtree>   // lazy segment tree
using namespace atcoder;
```

Option 2 — Local setup (download ACL)

1. Download ACL from: https://github.com/atcoder/ac-library
2. Extract — you get a folder called ac-library/
3. Compile with: g++ -I path/to/ac-library your_file.cpp

Option 3 — Competitive Companion / expander

Use the ACL expander script to bundle all headers into one file for submission on other judges.

Plain Segment Tree — atcoder/segtree

Supports **point updates** and **range queries** in O(log n).

The 3 Things You Must Define

Symbol	Type	Purpose	Rule it must satisfy
S	any type	Type of each node's value (e.g. int, long long, struct)	—
op(a,b)	S × S → S	How to merge two child nodes into a parent	Must be associative: op(op(a,b),c) == op(a,op(b,c))
e()	→ S	Identity element for op ("empty range" value)	op(e(), x) == op(x, e()) == x for all x

Instantiation

```
segtree<S, op, e> seg(n);          // size n, all initialised to e()
segtree<S, op, e> seg(vector);     // build from existing vector
```

Key Operations

Operation	Syntax	Complexity	Note
Point set	`seg.set(i, x)`	O(log n)	0-indexed
Point get	`seg.get(i)`	O(1)	
Range query	`seg.query(l, r)`	O(log n)	Half-open [l, r)

Operation	Syntax	Complexity	Note
All query	`seg.all_query()`	O(1)	Entire array
max_right	`seg.max_right(l, f)`	O(log n)	Rightmost r: f(query(l,r))==true
min_left	`seg.min_left(r, f)`	O(log n)	Leftmost l: f(query(l,r))==true

■ All ranges are half-open [l, r) — r is excluded.

Properties op Must Satisfy

Property	Requirement	Breaks without it
Associativity	`op(op(a,b),c) == op(a,op(b,c))`	Cannot merge children into parent correctly
Identity e()	`op(e(), x) == op(x, e()) == x`	Tree initialisation and empty-range queries are wrong

Quick Example — Range Min Query

```
using S = long long;
const S INF = 1e18;

S   op(S a, S b) { return min(a, b); }
S   e()          { return INF; }

segtree<S, op, e> seg(n);
seg.set(i, val);
seg.query(l, r);   // min in [l, r)
```

Lazy Segment Tree — atcoder/lazysegtree

Supports **range updates** AND **range queries** in O(log n). Each node stores two fields: **value (S)** and **lazy tag (F)**. It is one tree — not two.

The 7 Things You Must Define

Symbol	Side	Purpose	Identity rule
S	Value	Type of each node's stored value	—
op(a,b)	Value	Merge two node values (same as plain seg tree)	Must be associative
e()	Value	Identity for op ("empty range")	`op(e(),x) == x`
F	Lazy	Type of a lazy tag / pending update	—
mapping(f,x)	Both	Apply update f to node value x	`mapping(id(),x) == x`
composition(f,g)	Lazy	Collapse two tags: f applied after g	composition(id(),f)==f composition(f,id())==f
id()	Lazy	Identity tag — "no pending update"	`mapping(id(),x) == x`

Instantiation

```
lazy_segtree<S, op, e, F, mapping, composition, id> seg(n);
lazy_segtree<S, op, e, F, mapping, composition, id> seg(vector);
```

Key Operations

Operation	Syntax	Complexity	Note
Point set	`seg.set(i, x)`	O(log n)	
Point get	`seg.get(i)`	O(log n)	Pushes lazy down first
Point apply	`seg.apply(i, f)`	O(log n)	
Range apply	`seg.apply(l, r, f)`	O(log n)	Half-open [l, r)
Range query	`seg.query(l, r)`	O(log n)	Half-open [l, r)
All query	`seg.all_query()`	O(1)	
max_right	`seg.max_right(l, f)`	O(log n)	Binary search rightward
min_left	`seg.min_left(r, f)`	O(log n)	Binary search leftward

The 5 Properties That Must Hold

Property	Formal requirement	What breaks without it
op is associative	`op(op(a,b),c) == op(a,op(b,c))`	Cannot merge children into parent
e() is identity for op	`op(e(),x) == x`	Tree initialisation wrong
mapping distributes over op	`mapping(f,op(a,b)) == op(mapping(f,a),mapping(f,b))`	Lazy tag at parent gives wrong answer — silent bug
Updates compose	`mapping(composition(f,g),x) == mapping(f,mapping(g,x))`	Cannot defer updates; must always push down immediately

Property	Formal requirement	What breaks without it
id() is identity for mapping	`mapping(id(),x) == x`	Unmodified nodes get corrupted on pushdown

Note on terminology: The property 'updates compose' has no single standard word. In competitive programming people say 'the lazy tags are composable' or 'updates compose'. The formal structure is that lazy tags form a **monoid under composition**.

Common Update × Query Combinations

Query	Update	S	F	Works ?	Why
Sum	Add	`{sum, sz}`	`ll`	✓	Distributes linearly; size fixes it
Min/Max	Add	`ll`	`ll`	✓	min(a+f,b+f) = min(a,b)+f
Min/Max	Assign	`ll`	`optional<ll>`	✓	Trivially distributes
Sum	Assign	`{sum, sz}`	`optional<ll>`	✓	f * size
Sum	Multiply	`{sum, sz}`	`ll`	✓	(a*f)+(b*f) = (a+b)*f
Sum	Affine ax+b	`{sum, sz}`	`{a, b}`	✓	Compose: f(g(x))=(fa*ga)x+(fa*gb+fb)
Product	Subtract	`ll`	`ll`	✗	(a-f)(b-f) produces cross terms — distribution fails

Quick Checklist Before Coding

#	Question	What to verify
1	What is S?	What does each node store?
2	What is op?	How do I merge two nodes?
3	What is e()?	Identity — op(e(),x) == x
4	What is F?	What does a lazy tag look like?
5	What is mapping?	How does F change S? Does it distribute over op?
6	What is composition?	f after g = ? Can two tags always collapse into one?
7	What is id()?	mapping(id(),x) == x for all x

If questions 1–3 cannot be answered cleanly → plain segment tree may not work.
If question 5 (distribution) or 6 (composition) fails → lazy segment tree cannot be used directly.